NU

FAST National University of Computer and Emerging Sciences

Karachi Campus
Department of Computer Science

FINAL PROJECT REPORT

DictatorScript Compiler

A Custom Transpiling Programming Language and Multi-Stage Compiler
Built in C++ Using Encapsulation, Inheritance, Polymorphism, and the Visitor Pattern

Course:	Object Oriented Programming (OOP)
Instructor:	Sir Muhammad Khalid Khan
Section:	BAI-2C
Semester:	Spring 2026
Submission Date:	30 April 2026
Group Members	
Member 1 (Lead):	Taher Mustansir **25K-0119**
Member 2:	Noman Kamran **25K-0110**
Member 3:	Manaam Siddique **25K-0135**

Table of Contents

1. Executive Summary **3**

2. Introduction **3**

2.1 Background & Motivation 3

2.2 Problem Statement 3

2.3 Objectives 4

2.4 Scope 4

3. System Analysis **4**

3.1 Functional Requirements 4

3.2 Non-Functional Requirements 5

4. Object-Oriented Design **5**

4.1 Class Hierarchy & Pipeline Diagram 5

4.2 Class Descriptions 6

4.3 Relationships Between Classes 7

5. Implementation **7**

5.1 OOP Concepts Implemented 7

5.2 Code Snippets 8

5.3 File Structure 11

6. Testing & Results **11**

7. Challenges & Solutions **12**

8. Division of Work **12**

9. Conclusion & Future Work **13**

10. References **13**

1. Executive Summary

This report is about our OOP lab project — a custom language called **DictatorScript** and its compiler **dsc**, submitted to FAST-NUCES Karachi under Sir Muhammad Khalid Khan.

Our group wanted to do something different. Instead of building a normal management system like most groups do, we designed our own programming language and then wrote a compiler for it in C++. The way it works — you write code in DictatorScript, save it as a `.ds` file, run the compiler, and it gives you a working executable. Under the hood, it first converts `.ds` code into C++ and then calls g++ to compile that. This process is called transpiling. The reason this project fits so well with the OOP course is that every single OOP concept — encapsulation, inheritance, polymorphism, abstract classes, the Visitor design pattern, composition — comes up naturally just from building a compiler. On top of the compiler itself, we also wrote a VS Code extension so `.ds` files get colour-coded keywords in the editor. Our three-person team each took specific pieces of the compiler to build and own.

2. Introduction

2.1 Background & Motivation

When we were choosing a project topic, we thought about what would actually teach us something useful. We all write code every day but none of us really knew what happens between typing code and running a program. How does text become machine instructions? We wanted to find out, and building a compiler was the only real way to do that.

We also felt that most OOP projects we had seen from seniors were things like library systems or hotel booking apps. Those are fine, but OOP in those projects is mostly optional — you could write them without classes and they would still work. With a compiler, that is not

possible. A compiler genuinely needs separate classes for each stage, it genuinely needs inheritance for the AST nodes, it genuinely needs polymorphism for the Visitor pattern. OOP is not decorative here — it is necessary. That is what made us pick this. On top of that, we added a theme: all DictatorScript keywords use authoritarian language. Loops "impose," functions give "commands," conditions are "interrogated." It made the whole thing more fun to work on and easier to remember.

2.2 Problem Statement

There were a few problems we ran into when thinking about this project:

- Most OOP lab projects only use OOP in a surface-level way. We wanted a project where OOP is truly necessary, not just optional.
- Students rarely get to see how a compiler works from the inside. There are no simple, well-structured examples written at a beginner-to-intermediate level.
- Writing a compiler in a messy, unorganized way leads to code that breaks easily. Proper class separation is the only way to keep things manageable.
- There is no easy way to write DictatorScript in a code editor and get immediate visual feedback, such as colored keywords — so we had to build that too.

2.3 Objectives

1. Design a complete custom programming language — DictatorScript — with its own syntax, keywords, data types, and control flow.
2. Build a working multi-stage compiler (Lexer → Parser → Semantic Analyzer → Code Generator) where each stage is a separate, well-encapsulated C++ class.
3. Apply core OOP concepts throughout: encapsulation, abstraction, inheritance, polymorphism, and the Visitor design pattern.

4. Create a VS Code extension (dictatorscript-vscode) in TypeScript that highlights DictatorScript syntax.
5. Compile real DictatorScript programs — hello world, loops, functions, structs, pointers — into native executables.

2.4 Scope

- **In Scope:** Full compiler pipeline (Lexer, Parser, Semantic Analyzer, Code Generator), support for variables, functions, loops, conditionals, structs, pointers, arrays, maps, file imports, and a VS Code syntax extension.
- **Out of Scope:** Compiler optimizations, a graphical IDE, a debugger, or a standard library beyond basic I/O (planned as future work).

3. System Analysis

3.1 Functional Requirements

ID	Requirement	Priority
FR-01	Lexer reads a .ds source file and produces a stream of typed tokens, including line numbers for error reporting.	High
FR-02	Parser consumes the token stream and builds a complete Abstract Syntax Tree (AST) using recursive descent.	High
FR-03	Semantic Analyzer traverses the AST and detects errors: undeclared variables, type mismatches, duplicate declarations.	High
FR-04	Code Generator walks the validated AST and emits correct, compilable C++ source code.	High
FR-05	Compiler orchestrator invokes g++ on the generated .cpp file to produce a native executable.	High
FR-06	ErrorReporter collects errors across all stages and reports them with precise line numbers after each stage.	High
FR-07	CLI supports flags: --emit-cpp, --dump-tokens, --dump-ast, --verbose, and -o for custom output name.	Medium
FR-08	Language supports variables, functions, if/else, for/while loops, structs, pointers, arrays, and maps.	High

FR-09	Multi-file import using the `import "filename"` keyword is supported.	Medium
FR-10	VS Code extension provides syntax highlighting for all DictatorScript keywords and constructs in .ds files.	Low

3.2 Non-Functional Requirements

- **Code Quality:** Each compiler stage must be a separate class following the Single Responsibility Principle. Header (.h) and source (.cpp) files must be separate for all classes.
- **Robustness:** The compiler must handle malformed source files gracefully — it should report clear error messages rather than crashing.
- **Maintainability:** The Visitor Pattern must be used in the Semantic Analyzer and Code Generator stages so that adding a new compiler pass later requires only a new Visitor class, not changes to the AST.
- **Portability:** The compiler must build and run on Windows (MSVC, MinGW), Linux, and macOS using CMake as the build system.
- **Performance:** Compiling a typical DictatorScript file of up to 500 lines should complete in under one second on a modern machine.

4. Object-Oriented Design

4.1 Class Hierarchy & Pipeline Diagram

We broke the compiler into five stages, each one its own class. The output of one stage becomes the input of the next. Below is a rough diagram showing how data flows through the system and how the main classes relate to each other:

```
                    — COMPILER PIPELINE
 ———————————————————————————————————— .ds source
   file | [ Lexer ] reads text → produces Token
  stream | [ Parser ] reads tokens → builds AST |
 [ SemanticAnalyzer ] walks AST → validates types &
  scopes (Visitor) | [ CodeGenerator ] walks AST →
```

```
emits C++ source code (Visitor) | [ .cpp file ] |
    [ g++ subprocess ] compiles .cpp → native
        executable ── CLASS RELATIONSHIPS
  ──────────────────────────────────── ASTNode
<<abstract>> ├── ProgramNode ├── VarDeclNode ├──
FunctionNode ├── IfNode ├── ForNode / WhileNode
├── BinaryExprNode ├── CallExprNode └── ... (all
   other node types) Visitor <<abstract>> ├──
   SemanticAnalyzer └── CodeGenerator Compiler
 ──composes──> Lexer, Parser, SemanticAnalyzer,
          CodeGenerator, ErrorReporter
```

4.2 Class Descriptions

Lexer

The Lexer is where everything starts. It opens the `.ds` file and reads it one character at a time. As it reads, it groups characters together into things called tokens — for example, the word `interrogate` gets turned into a keyword token, and a number like `42` becomes an integer token. Every token also carries the line number it was on, which helps us give useful error messages later. Comments — both single-line `//` and multi-line `/* */` — are just ignored and skipped.

Token & TokenType

A Token is basically just a small container. It holds three pieces of info: the type of token it is (like KEYWORD or IDENTIFIER or INTEGER_LITERAL), the actual text that was scanned, and the line it came from. TokenType is just an enum that lists all the possible categories. We kept Token separate from the Lexer so that the Parser could work with a clean list of tokens without worrying about how they were produced.

Parser

After the Lexer is done, we have a flat list of tokens. The Parser's job is to turn that flat list into a tree — an Abstract Syntax Tree or AST. The tree captures the structure of the program. For instance, an

`interrogate` block becomes an IfNode in the tree, with separate child nodes for the condition and the body. We used recursive descent parsing, which means there is one function per grammar rule and they call each other to handle nesting.

ASTNode (Abstract Base Class)

ASTNode is the base class that all tree nodes inherit from. It has one pure virtual method — `accept(Visitor&)` — which is what makes the Visitor Pattern possible. When any part of the compiler calls `accept()` on a node, the node turns around and calls the right `visit()` method on whoever called it. Since ASTNode is abstract, you cannot make one directly. Every node in the tree is always a real, specific type like IfNode, FunctionNode, or VarDeclNode.

SemanticAnalyzer

Once the tree is built, we need to check if the program actually makes sense — not just if it is written correctly, but if it is logically valid. That is the SemanticAnalyzer's job. It checks things like: did the programmer use a variable they never declared? Are they passing the wrong number of arguments to a function? Are they doing something invalid like adding a string and an integer? It does all this by visiting each node using the Visitor Pattern and calling `visit()` for every node type. Any problems go to the ErrorReporter.

SymbolTable

The SymbolTable is used by the SemanticAnalyzer to keep track of what has been declared where. It works with scopes — when we enter a block like a loop body, a new scope layer is added on top. When we leave, that layer is removed. This way, a variable declared inside a loop cannot accidentally be used outside it. Internally it is just a stack of maps.

CodeGenerator

The CodeGenerator is another Visitor, just like the SemanticAnalyzer. But instead of checking for errors, it walks the same tree and for each node it writes out the matching C++ code into a string stream. An

IfNode becomes an `if (...) { }` block. A FunctionNode becomes a proper C++ function. When it is done, the whole output string is saved as a `.cpp` file ready to be compiled.

ErrorReporter

ErrorReporter is a shared object that all stages have access to. Instead of crashing the moment something goes wrong, each stage reports its errors through this object. The errors accumulate with their line numbers, and at the end they are all printed together. This is much better than stopping at the first issue — it lets the user see and fix multiple problems in one go.

Compiler

The Compiler class ties everything together. It creates and owns all the other stage objects and runs them one by one — Lexer, then Parser, then SemanticAnalyzer, then CodeGenerator. After code generation it calls g++ on the produced `.cpp` file. From the outside, the user just types `dsc myfile.ds` and gets an executable. The Compiler also reads the command-line flags and decides what to do with them.

4.3 Relationships Between Classes

Relationship	Classes Involved	Type
All AST nodes inherit from ASTNode	ASTNode, IfNode, ForNode, FunctionNode, VarDeclNode, etc.	Inheritance (IS-A)
SemanticAnalyzer and CodeGenerator are both Visitors	Visitor (abstract), SemanticAnalyzer, CodeGenerator	Inheritance (IS-A)
Compiler owns all pipeline stage objects	Compiler, Lexer, Parser, SemanticAnalyzer, CodeGenerator	Composition (HAS-A)
ErrorReporter is shared across all stages	Compiler, Lexer, Parser, SemanticAnalyzer, CodeGenerator, ErrorReporter	Dependency / Shared Reference
Parser produces AST consumed by Analyzer and Generator	Parser, SemanticAnalyzer, CodeGenerator	Association

SymbolTable is used by SemanticAnalyzer	SemanticAnalyzer, SymbolTable	Composition (HAS-A)

5. Implementation

5.1 OOP Concepts Implemented

OOP Concept	Implementation Detail
Encapsulation	All internal details of each stage are kept private. Nobody outside the Lexer can see its character buffer. Nobody outside the Parser can touch its token index. Only the public interface methods — tokenize(), parse(), analyze() — are visible to the outside world.
Abstraction	ASTNode is abstract. The compiler stages just work with ASTNode pointers and never need to know what specific type of node they are dealing with. The Visitor Pattern takes care of figuring out the right method to call at runtime.
Inheritance	There are two inheritance chains here. First, all AST node types — IfNode, ForNode, VarDeclNode, FunctionNode, etc. — inherit from ASTNode. Second, SemanticAnalyzer and CodeGenerator both inherit from an abstract Visitor base class.
Polymorphism	Each node subclass implements `accept(Visitor& v)`, which calls `v.visit(*this)`. At runtime C++ picks the right `visit()` overload based on the actual Visitor object — either the analyzer or the generator. This is runtime polymorphism working as intended.
Abstract Classes	Both ASTNode and Visitor are abstract — they each have at least one pure virtual method and cannot be instantiated. This forces all subclasses to provide their own implementations.
Visitor Design Pattern	This was the most important pattern in our project. It keeps the AST structure completely separate from whatever we want to do with it. Adding a new pass — say, an optimizer — only means writing a new Visitor class. The AST nodes do not need to change at all.
Composition	The Compiler class owns a Lexer, Parser, SemanticAnalyzer, CodeGenerator, and ErrorReporter as member objects. It does not inherit from any of them. This is a deliberate choice — composition made more sense than inheritance here.
Single Responsibility	Each class has exactly one job and does not step outside it. The Lexer only tokenizes. The Parser only builds the tree. The Analyzer only validates. The Generator only outputs C++. No class ever tries to do two things at once.

Error Handling	We created a shared ErrorReporter and passed it to every stage. When something goes wrong, the stage reports the error with its line number and continues. All errors are printed at the end, so the user sees everything at once rather than fixing one problem and recompiling just to see the next one.
STL Containers	We used `std::vector` for the token list and for storing child nodes inside parent AST nodes. `std::unordered_map` is used in the SymbolTable to look up variable types by name. `std::string` handles all text throughout the compiler.
Constructors & Destructors	All node classes have constructors that set up their fields right away. ASTNode has a virtual destructor, which means when the tree is deleted starting from the root, every child node gets properly cleaned up even through pointers of the base type.

5.2 Code Snippets

Abstract Base Class: ASTNode

```
// ASTNode.h
#pragma once
#include <string>
#include <vector>
using namespace std;

// Forward-declare Visitor so ASTNode can reference it
class Visitor;

class ASTNode {
public:
    int lineNumber;   // Source line — used for error
messages

    ASTNode(int line) : lineNumber(line) {}

    // Pure virtual — every node must implement accept()
    // This is what makes the Visitor Pattern work
    virtual void accept(Visitor& visitor) = 0;

    virtual ~ASTNode() {}
};

// Example concrete node: variable declaration
// "declare int x = 5"  becomes one of these
class VarDeclNode : public ASTNode {
public:
    string typeName;    // "int", "float", "string",
```

```
etc.
    string varName;     // the name the user chose
    ASTNode* initializer; // the right-hand side, or
nullptr
    bool isConst;         // true if "declare elite"

    VarDeclNode(int line, string type, string name,
                ASTNode* init, bool cnst)
        : ASTNode(line), typeName(type), varName(name),
          initializer(init), isConst(cnst) {}

    void accept(Visitor& visitor) override;

    ~VarDeclNode() { delete initializer; }
};
```

Visitor Pattern: Abstract Visitor and CodeGenerator

```
// Visitor.h  — abstract base, one visit() per node type
#pragma once
class VarDeclNode;
class IfNode;
class ForNode;
class FunctionNode;
class BinaryExprNode;
class CallExprNode;
// ... forward declarations for all node types

class Visitor {
public:
    virtual void visit(VarDeclNode& node)    = 0;
    virtual void visit(IfNode& node)         = 0;
    virtual void visit(ForNode& node)        = 0;
    virtual void visit(FunctionNode& node)   = 0;
    virtual void visit(BinaryExprNode& node) = 0;
    virtual void visit(CallExprNode& node)   = 0;
    // ... one pure virtual per node type
    virtual ~Visitor() {}
};

// CodeGenerator.h — inherits Visitor, emits C++ for
each node
#include "Visitor.h"
#include <sstream>

class CodeGenerator : public Visitor {
private:
```

```
    ostringstream output;   // builds the C++ source as
a string
    int indentLevel = 0;

    string indent() const {
        return string(indentLevel * 4, ' ');
    }

public:
    // Called when the Visitor reaches a variable
declaration node
    void visit(VarDeclNode& node) override {
        output << indent();
        if (node.isConst) output << "const ";
        output << node.typeName << " " << node.varName;
        if (node.initializer) {
            output << " = ";
            node.initializer->accept(*this);
        }
        output << ";\n";
    }

    // Called when the Visitor reaches an if-statement
node
    void visit(IfNode& node) override {
        output << indent() << "if (";
        node.condition->accept(*this);
        output << ") {\n";
        indentLevel++;
        for (ASTNode* stmt : node.body) stmt-
>accept(*this);
        indentLevel--;
        output << indent() << "}\n";
        if (node.elseBody) {
            output << indent() << "else {\n";
            indentLevel++;
            node.elseBody->accept(*this);
            indentLevel--;
            output << indent() << "}\n";
        }
    }

    string getOutput() const { return output.str(); }
};
```

Lexer: Tokenizing DictatorScript Source

```
// Lexer.cpp  — simplified illustration of keyword
scanning
#include "Lexer.h"

// Maps DictatorScript keywords to their TokenType
const unordered_map<string, TokenType> Lexer::keywords =
{
    { "regime",       TokenType::KW_REGIME      },
    { "declare",      TokenType::KW_DECLARE      },
    { "elite",        TokenType::KW_ELITE        },
    { "command",      TokenType::KW_COMMAND      },
    { "report",       TokenType::KW_REPORT       },
    { "interrogate",  TokenType::KW_INTERROGATE  },
    { "otherwise",    TokenType::KW_OTHERWISE    },
    { "impose",       TokenType::KW_IMPOSE       },
    { "forever",      TokenType::KW_FOREVER      },
    { "silence",      TokenType::KW_SILENCE      },
    { "proceed",      TokenType::KW_PROCEED      },
    { "broadcast",    TokenType::KW_BROADCAST    },
    { "demand",       TokenType::KW_DEMAND       },
    { "loyal",        TokenType::KW_LOYAL        },
    { "traitor",      TokenType::KW_TRAITOR      },
    { "law",          TokenType::KW_LAW          },
    { "summon",       TokenType::KW_SUMMON       },
    { "kill",         TokenType::KW_KILL         },
    { "import",       TokenType::KW_IMPORT       },
};

Token Lexer::scanIdentifierOrKeyword() {
    string word;
    while (pos < source.size() &&
           (isalnum(source[pos]) || source[pos] == '_'))
{
        word += source[pos++];
    }
    // Check if the word is a known keyword
    auto it = keywords.find(word);
    if (it != keywords.end())
        return Token(it->second, word, line);
    return Token(TokenType::IDENTIFIER, word, line);
}
```

Semantic Analyzer: Checking Variable Declarations

```
// SemanticAnalyzer.cpp — visit() for variable
declaration
#include "SemanticAnalyzer.h"

void SemanticAnalyzer::visit(VarDeclNode& node) {
    // Check that this name has not already been
declared
    // in the current scope
    if (symbolTable.existsInCurrentScope(node.varName))
{
        errorReporter.report(
            node.lineNumber,
            "Variable '" + node.varName + "' is already
declared."
        );
        return;
    }

    // If there is an initializer, analyze it first
    if (node.initializer) {
        node.initializer->accept(*this);

        // Type check: make sure the initializer type
matches
        string initType = lastExprType;
        if (!typesMatch(node.typeName, initType)) {
            errorReporter.report(
                node.lineNumber,
                "Type mismatch: cannot assign " +
initType +
                " to " + node.typeName
            );
        }
    }

    // Register the variable in the current scope
    symbolTable.declare(node.varName, node.typeName,
node.isConst);
}
```

5.3 File Structure

File	Purpose
main.cpp	

	Entry point; parses CLI arguments and starts the Compiler
Compiler.h / Compiler.cpp	Pipeline orchestrator; owns and coordinates all stages
ErrorReporter.h / ErrorReporter.cpp	Shared error collection and reporting service
Token.h / Token.cpp	Token data class (type, text, line number)
TokenType.h	Enum listing every token type the Lexer can produce
Lexer.h / Lexer.cpp	Stage 1: tokenizes .ds source text
ASTNode.h / ASTNode.cpp	Abstract base class and all concrete AST node subclasses
Parser.h / Parser.cpp	Stage 2: recursive descent parser; builds the AST
SymbolTable.h / SymbolTable.cpp	Scoped symbol table for variable and function tracking
SemanticAnalyzer.h / SemanticAnalyzer.cpp	Stage 3: Visitor that validates types and scopes
CodeGenerator.h / CodeGenerator.cpp	Stage 4: Visitor that emits C++ source code
CMakeLists.txt	Cross-platform build configuration (CMake 3.16+)
dictatorscript-vscode/ (TypeScript)	VS Code extension with TextMate grammar for .ds syntax highlighting
examples/*.ds	Working example DictatorScript programs

6. Testing & Results

6.1 Functional Test Cases

Test Case	Input / Action	Expected Result	Status
Hello World program	Compile hello_world.ds	Executable prints "Hello, World!" correctly	Pass
Variable declaration	declare int x = 5	Emits: int x = 5; in C++ output	Pass
Const variable (elite)	declare elite int x = 10	Emits: const int x = 10;	Pass

Function definition	command add(int a, int b) -> int	Emits correct C++ function signature and body	Pass
Range-based for loop	impose (int i from [1..5])	Emits: for (int i = 1; i <= 5; i++)	Pass
While loop	impose (x > 0)	Emits: while (x > 0) { ... }	Pass
Conditional statement	interrogate / otherwise	Emits correct if / else if / else chain	Pass
Struct definition	law Point { int x; int y; }	Emits: struct Point { int x; int y; };	Pass
Dynamic memory	summon Point / kill ptr	Emits: new Point / delete ptr correctly	Pass
Undeclared variable error	Use variable before declare	SemanticAnalyzer reports error with line number	Pass
Duplicate declaration	Declare same name twice in same scope	Error: "already declared in this scope"	Pass
Type mismatch	Assign string value to int variable	SemanticAnalyzer reports type mismatch	Pass
--emit-cpp flag	dsc sample.ds --emit-cpp	Produces sample.cpp but does not invoke g++	Pass
--dump-tokens flag	dsc sample.ds --dump-tokens	Prints full token stream to console	Pass
Multi-file import	import "utils" in main.ds	utils.ds is compiled and linked correctly	Pass

6.2 Sample Output

[Screenshot: Terminal — dsc compiling hello_world.ds, then running the produced executable]

[Screenshot: --dump-tokens output showing keyword, identifier, and literal tokens with line numbers]

[Screenshot: Semantic error output — "Line 7: Variable 'total' used before declaration"]

[Screenshot: VS Code editor showing .ds file with colored keywords, types, and strings via the extension]

7. Challenges & Solutions

Challenge	Solution Applied
Visitor Pattern requires every Visitor class to have a visit() method for every single node type. Adding a new node type means updating every Visitor.	We designed the full set of node types carefully before writing any Visitor code, so the interface was stable. A forward-declaration pattern in Visitor.h kept compile times manageable.
The Parser needed to handle operator precedence correctly (e.g., multiplication before addition) without getting into an infinite loop.	We used the standard recursive descent technique with separate methods per precedence level: parseAddition() calls parseMultiplication(), which calls parseUnary(), and so on.
Memory management for the AST — every node is heap-allocated and the tree can be hundreds of nodes deep.	Every parent node owns its children and deletes them in its destructor. The root ProgramNode deletes everything when it goes out of scope, so there are no leaks.
The SymbolTable needed to support nested scopes properly, including the case where a variable in an inner scope shadows one in an outer scope.	Implemented as a stack of unordered_maps. When entering a block, a new map is pushed; when leaving, it is popped. Lookup walks the stack from top to bottom.
Generating correct C++ for pointer syntax — DictatorScript uses `int->` for pointer types and `^` for dereference, which are very different from C++.	The CodeGenerator's visit() for pointer-related nodes maps these directly: `int->` becomes `int*`, `^ptr` becomes `*ptr`, and `@x` becomes `&x`.
The VS Code extension required learning TypeScript and the TextMate grammar format, which none of us had used before.	We studied existing grammar files for similar languages and adapted the pattern. The extension uses a JSON-based grammar file (tmLanguage) which maps regex patterns to color scopes.

8. Division of Work

Member	Roll No.	Modules / Tasks	Contribution
Taher Mustansir	25K-0119	Compiler class (pipeline orchestrator), full Parser implementation (all grammar rules), CodeGenerator (C++ emission for all node types), CLI argument handling in main.cpp, CMakeLists.txt build system, final integration and testing	34%
Noman Kamran	25K-0110	Lexer implementation and full keyword map, Token and TokenType design, SemanticAnalyzer (type checking and scope validation), SymbolTable with nested scope support, ErrorReporter design, debugging of edge cases	33%
Manaam Siddique	25K-0135	Full ASTNode class hierarchy (all concrete node types), VS Code syntax-highlighting extension in TypeScript (dictatorscript-vscode), test suite and all example .ds programs, project report and documentation	33%

9. Conclusion & Future Work

9.1 Conclusion

Honestly, this was probably the hardest project any of us has done at FAST so far. We started with literally nothing — no language, no compiler, just an idea — and by the end we had something that actually works. You can write a DictatorScript program, compile it, and run it. That still feels a bit unreal to us.

What surprised us the most was how naturally the OOP concepts fit in. We did not sit down and say "let us use encapsulation here." It just happened, because it is the only way to keep a project this size from falling apart. Each stage had to be its own class, the AST had to be a

proper inheritance hierarchy, the Visitor Pattern was the only clean way to handle the analyzer and code generator without making a mess. We genuinely used these concepts because we needed them, not to fulfill a checklist.

All three of us went through the entire codebase, not just our own parts. We spent time explaining things to each other and fixing each other's bugs. We can answer questions about any part of it during the viva.

9.2 Future Work

- Right now you have to manually include C++ headers for things like math. We want to add a DictatorScript standard library so basic things like math functions and string operations are built in.
- We want to add an optimizer pass — basically a new Visitor that goes through the tree before code generation and simplifies things like constant expressions.
- DictatorScript only supports structs right now. It would be interesting to add actual classes so you can write OOP-style DictatorScript programs.
- The VS Code extension is only installed locally. We want to publish it to the official VS Code Marketplace so anyone can use it.
- Currently the compiler calls g++ as a subprocess to compile the generated C++ file. A cleaner approach would be to generate LLVM IR directly, which would remove the dependency on g++ being installed.
- Testing right now is mostly manual. We want to set up automated tests using Google Test so every stage can be tested independently.

10. References

1. Stroustrup, B. *The C++ Programming Language, 4th Edition.* Addison-Wesley, 2013.
2. Lippman, S., Lajoie, J., Moo, B. *C++ Primer, 5th Edition.* Addison-Wesley, 2012.

3. Aho, A., Lam, M., Sethi, R., Ullman, J. *Compilers: Principles, Techniques, and Tools (The Dragon Book), 2nd Edition.* Pearson, 2006.
4. Nystrom, R. *Crafting Interpreters.* Available free online at https://craftinginterpreters.com/
5. cppreference.com. *C++ Reference — Classes, Templates, STL, Virtual Functions.* https://en.cppreference.com/
6. Gamma, E., Helm, R., Johnson, R., Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software.* Addison-Wesley, 1994. (Visitor Pattern reference)
7. Visual Studio Code. *Syntax Highlight Guide — TextMate Grammars.* https://code.visualstudio.com/api/language-extensions/syntax-highlight-guide
8. Course Lab Materials — Sir Muhammad Khalid Khan, FAST-NUCES Karachi, OOP Lab, Spring 2026.

Group Representative	Instructor
Taher Mustansir — 25K-0119	Sir Muhammad Khalid Khan